UniNet: A Unified Multi-Granular Traffic Modeling Framework for Network Security

Binghui Wu, *Graduate Student Member, IEEE*, Dinil Mon Divakaran, *Senior Member, IEEE*, and Mohan Gurusamy, *Senior Member, IEEE*

***Abstract*—As modern networks grow increasingly complex—driven by diverse devices, encrypted protocols, and evolving threats—network traffic analysis has become critically important. Existing machine learning models often rely only on a single representation of packets or flows, limiting their ability to capture the contextual relationships essential for robust analysis. Furthermore, task-specific architectures for supervised, semi-supervised, and unsupervised learning lead to inefficiencies in adapting to varying data formats and security tasks.**

To address these gaps, we propose UniNet, a unified framework that introduces a novel multi-granular traffic representation (T-Matrix) with rich contextual information, integrating session, flow, and packet-level features to provide comprehensive contextual information. Combined with T-Attent, a specially designed lightweight attention-based model, UniNet efficiently learns latent embeddings for diverse security tasks. Extensive evaluations across four key network security and privacy problems—anomaly detection, attack classification, IoT device identification, and encrypted website fingerprinting—demonstrate UniNet's significant performance gain over state-of-the-art methods, achieving higher accuracy, lower false positive rates, and improved scalability across various datasets. By addressing the limitations of single-level models and unifying traffic analysis paradigms, UniNet sets a new benchmark for modern network security.

***Index Terms*—Network security, network traffic analysis, anomaly detection, website fingerprinting, representation learning, machine learning, multi-granular modeling, unified model.**

I. Introduction

OVER the years, computer networks have evolved significantly due to the increase in network bandwidth, sophisticated network nodes (such as programmable switches), new device types (e.g., Internet of Things), changing network protocols (e.g., DNS-over-HTTPS), new applications (e.g., ChatGPT), etc. With this evolution also comes the challenge of securing the networks from various threats and attacks. Traditional rule-based systems have limitations in catching up with new and unknown threats; moreover, payloads are not available for deep packet inspection due to the increasing adoption of TLS [1]. Consequently, researchers have long been exploring models from the domain of statistics, data mining, and machine learning (ML) to address the challenges in network traffic analysis [2], [3], [4], [5]. The advancement in deep learning (DL) plays a crucial role in network traffic analysis for security tasks. These models leverage the vast and complex features of network traffic to identify anomalies and threats effectively. Additionally, with the advent of programmable switches [6], there is potential for ML or partial ML logic to run directly on switches at terabits per second (Tbps) line rates [7], [8], promising real-time security capabilities. The deep learning models, from convolutional neural networks (CNNs) to autoencoders and the latest transformer models [9] are able to learn from large datasets consisting of hundreds of features. This has led to the development of several deep learning models for network anomaly detection, botnet detection, attack classification, fingerprinting and counter-fingerprinting of IoT devices and websites, traffic generation, and so on [10], [11], [12], [13], [14], [15].

Despite these promising directions, a core challenge lies in data representation and formats. The common formats for network data are: i) `pcap` that captures every packet on the wire and details from the headers ii) flows (e.g., NetFlow, IPFIX [16]) that capture coarser information from an aggregation of packets. Packet captures provide rich details but require substantial resources to store and process; flow-based representations are more lightweight but lose important per-packet granularity. As a result, ML models must adapt to different levels of detail and data availability. Traditional intrusion detection systems (IDS) often focus on flows only, treating each flow as an isolated unit [17], [18]. However, malicious behaviors rarely manifest in any single flow or packet in isolation. A single flow generally lacks conclusive evidence, and a lone packet offers minimal context unless considered within a broader temporal and relational environment. Therefore, recent efforts are shifting toward session-level representations, wherein flows sharing common attributes (e.g., source or destination IP addresses) within a certain time window are grouped into sessions. Session-level analysis provides more context than flow-level or packet-level views alone. However, most research works focus exclusively on one granularity at a time, which can either overlook subtle patterns critical for detecting sophisticated threats or demand excessive computational resources, undermining scalability and real-time applicability.

Recent research works have shown the ability to parse packets at line-rates for security use cases, e.g., rule-based

Received 12 March 2025; revised 30 April 2025 and 18 June 2025; accepted 23 June 2025. Date of publication 2 July 2025; date of current version 29 December 2025. The associate editor coordinating the review of this article and approving it for publication was G. Han. *(Corresponding author: Binghui Wu.)*

Binghui Wu and Mohan Gurusamy are with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore 117576 (e-mail: binghuiwu@u.nus.edu; gmohan@nus.edu.sg).

Dinil Mon Divakaran is with the Institute for Infocomm Research, A*STAR, Singapore 138632, and also with the School of Computing, National University of Singapore, Singapore (e-mail: dinil_divakaran@i2r.a-star.edu.sg).

Digital Object Identifier 10.1109/TCCN.2025.3585170

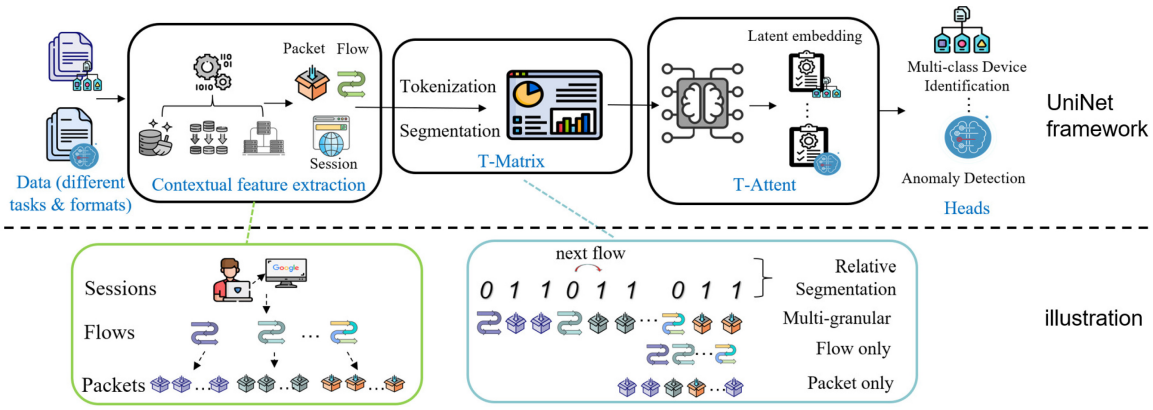


Fig. 1. Overview of UniNet framework.

TABLE I
EXTENDED COMPARISON WITH RECENT NETWORK TRAFFIC ANALYSIS MODELS

Model	Multi-granular input	Multi-task	Multi-learning paradigms
AutoWFP [20]	No	No	No
TMWF [21]	No	No	No
TDoQ [22]	No	No	Yes (Supervised, Semi-supervised)
SANE [14]	No	No	No
GRU-tFP [23]	No	Yes	Yes (Supervised, un-supervised)
BiLSTM-iFP [24]	No	No	No
MNTD [25]	Yes (Packet + Time)	No	No
UniNet (Ours)	**Yes (Session + Flow + Packet)**	**Yes**	**Yes (Supervised, Semi-, Unsupervised)**

TABLE II
TASKS WE CONSIDER FOR NETWORK TRAFFIC ANALYSIS (SEE THREAT MODEL DESCRIPTION IN SECTION V FOR FURTHER DETAILS)

Tasks	Learning paradigm	Granularity	Task ID
Anomaly detection	one-class un/semi-supervised	session, flow, packet	1
Attack identification	binary/multi-class supervised	flow, packet	2
Device identification	multi-class supervised	session, flow, packet	3
Website fingerprinting	multi-class semi-supervised	session, flow, packet	4

DDoS detection and mitigation [8], [19]. Yet, representing all packets (of a traffic session) in a model is challenging. Firstly, using full packet sequences as inputs to the model makes the model size too large to be trained efficiently. Additionally, longer input sequences increase the inference time and make the system more vulnerable to certain attacks, e.g., DoS attacks specifically targeting such models. ① Therefore, a key gap we identify in this domain is the lack of *efficient* and *effective* representation that includes both packet-, flow- and session-level features. Without such a representation, models are not easily adaptable for deployment across various networks that may have different traffic-capturing techniques and constraints, resulting in different data formats. ② Furthermore, another critical gap is the uncertainty around the type of model best suited to handle these challenges. A general model that can function effectively across different conditions is important, as it ensures consistent performance despite variability in available features. Such a model must be capable of integrating various learning paradigms, including supervised, semi-supervised, and unsupervised learning. ③ Beyond the need for a general representation and a unified model, there is also a challenge of dealing with limited data. In scenarios where data is scarce, the ability to extract meaningful information and maintain robust performance becomes important. Table I presents a comparison with recent traffic analysis models.

To address these limitations, we introduce UniNet, a unified framework designed to integrate multi-granular representations and support a broad range of network traffic analysis tasks. Figure 1 provides an overview of UniNet, highlighting its three main components: i) T-Matrix, A multi-granular traffic representation that can integrate session, flow, and packet level information; ii) T-Attent, A unified, self-attention-based feature extraction model capable of capturing contextual patterns from diverse data inputs; and iii) heads tailored to different learning paradigms, including supervised, semi-supervised, and unsupervised tasks. Unlike previous approaches that either focus on flows or packets in isolation, UniNet leverages these granularities in a single architecture, ensuring both fine-grained context and scalability. At the same time, its flexible architecture supports a variety of security and privacy tasks, from anomaly detection and attack classification to device identification and website fingerprinting (see Table II).

The following summarizes our contributions:

1) *T-Matrix:* We develop a multi-granular representation for network traffic that is suitable for multiple data formats and their combinations (Section III). We carry out comprehensive experiments to compare T-Matrix with single-level representations; the results show that T-Matrix captures more detailed traffic patterns, leading to improved performance in various traffic analysis tasks (Section V-D).
2) *T-Attent for Latent Embedding Learning:* We develop a unified attention-based architecture for network traffic analysis that captures contextual information and simplifies model selection (Section IV). T-Attent effectively handles supervised, semi-supervised, and unsupervised learning by employing different "heads" (Section IV-B). This design greatly reduces the overhead of using separate models for each task, making UniNet a powerful choice for diverse traffic analysis scenarios (Section V. Additionally, we adopt a lightweight variant of the transformer encoder and a new segmentation strategy (Section IV-A), with reduced attention heads and

embedding dimensions, which ensures computational efficiency without compromising performance.

3) *Enhanced efficiency and performance:* We evaluate UniNet on four common network security and privacy tasks spanning three ML categories (unsupervised anomaly detection, supervised classification of attacks and devices, and semi-supervised website fingerprinting), using multiple real-world datasets (Section V). UniNet consistently outperforms existing baselines in terms of detection rates and related metrics. Furthermore, we highlight ability of UniNet to discover intrinsic patterns from limited data (Section V-C). The self-attention mechanism in T-Attent shows significant advantages in extracting information from informative sequences compared to baselines. We publish our code base for supporting future research and reproducibility.[1]

II. UniNet Framework

We present an overview of our proposal, UniNet. As depicted in Fig. 1, UniNet operates in four key steps. i) The first step involves extracting semantic features at multiple levels, such as packet, flow, and session, to retain rich contextual information and meaningful fields; subsequently we define a multi-granular cohesive traffic representation T-Matrix (Section III). ii) In the second step, the unified T-Matrix representation is encoded into tokens for training the model. In Section III-A, we define the vocabulary of tokens corresponding to important traffic features and describe the tokenization process. iii) After encoding the T-Matrix representation of traffic into tokens, they are provided as input into the self-attention model, T-Attent, for representation learning. We propose a relative segmentation embedding in Section IV, which allows the model to identify and aggregate features at different levels, enhancing its ability to learn meaningful representations from the data. The output of T-Attent is a latent embedding that represents the understanding of the traffic. iv) This latent embedding is general enough to be used for various tasks, which is achieved by feeding it into different task-specific heads, as explained in Section IV-B. These heads provide a flexible framework for multiple network traffic analysis tasks.

III. T-Matrix Design

T-Matrix is a multi-granular traffic representation that encompasses information at three different levels of traffic information: session, flow, and packet. This is different from existing works that capture either flow-level or packet-level information but not both, thereby limiting the modeling capability. Incorporating lightweight domain knowledge is often necessary to extract meaningful patterns from raw network traffic. T-Matrix adopts basic yet generalizable features, such as port categories and TCP flag encodings, that are protocol-agnostic and widely validated in prior work [14], [22], [24], [26]. These features enable UniNet to generalize across diverse tasks and protocols, without heavy reliance on manual feature engineering. Traffic analysis systems typically operate by tapping traffic so that false positives (FPs), however low their number, do not interrupt normal connections. A stream of packets should be analyzed before decision-making. To support efficient analysis under this constraint, UniNet adopts a *sliding window* strategy. Rather than waiting for a full traffic to complete, the system buffers packets within a fixed-size time window and extracts features from it. We define *session* as a finite aggregation of *flows* that are temporally correlated and are contextualized by src/dst IP address. For example, a 15-minute traffic to and from one IP address forms a session. The separation of different sessions can be based on time (static) or based on inactivity (dynamic, e.g., 'a silence of 1-min breaks a session into two'). Each *flow* in a session is a set of packets identified by the common 5-tuple of src/dst IP address, src/dst ports, and protocol. Thus, a session represents the behavior of, say, a user's browsing activity over a short period of time; the flows in the session describe the various connections, such as DNS query/response, HTTP request/response to different servers for various resources to load a website, and so on. Fig. 2 illustrates the semantic multi-granular traffic representation of T-Matrix.

Per-packet features are obtained from fields in the packet header. The raw packet features useful for traffic analysis include packet size, time since the last packet, packet direction, packet direction (incoming/outgoing), transport protocol (TCP/UDP), application protocol (HTTP, DNS, NTP, etc.), TLS presence and version, the categories of source/destination IP addresses (internal/external) and ports (service port, in particular). The port number helps to determine the type of application traffic, specifically differentiating between service (well-known) ports and ephemeral (random) ports. However, a single packet alone might not provide sufficient information for traffic analysis. Packet-level features are meaningful when a sequence of packets is considered. For example, a TCP SYN packet is present in both benign and malicious flow; as it does not *independently* help in determining whether the packet (and the corresponding flow) is malicious. However, when we analyze a sequence of packets, we may observe a rare pattern that indicates an anomaly; e.g., repetitive sequences with identical packet sizes, which are characteristic of application-layer DDoS attacks. Therefore, we extract these features from sequences of packets, encoding them (Section III-A) to subsequently use the encoded features for training and inference. As payloads are (mostly) encrypted, we do not process payloads for feature extraction.

Flow-level features are aggregated from the headers of packets in a flow. This aggregation reduces the amount of data, but it is still useful when there are missing packet-level features due to resource limitations or when users tunnel through encrypted channels such as ToR and VPN. The identifier of a flow is the 5-tuple: src and dst IP addresses and port numbers, and transport protocol. Since data can flow in both directions, the forward and reverse flow identifiers are matched to learn the relationship. A silence period is used to determine the expiry of a 5-tuple flow within a session. There are tens of flow-level features that can be extracted from network traffic, and UniNet is designed to represent a

[1]Code is available at: `https://github.com/Binghui99/UniNet`.

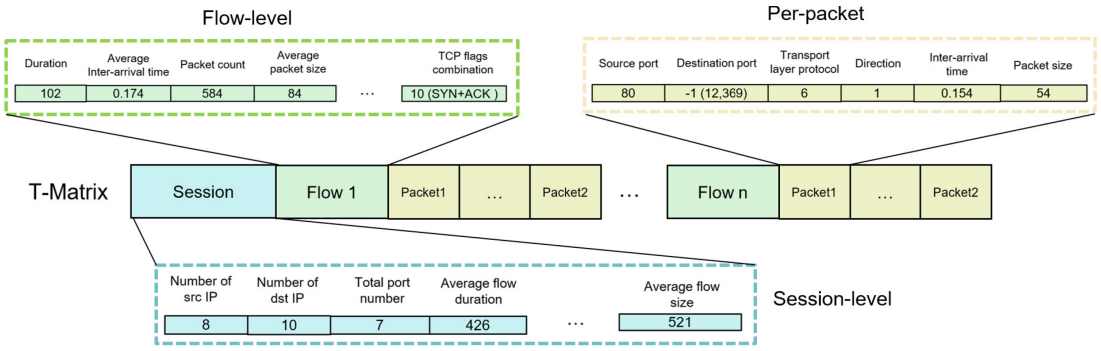


Fig. 2. T-Matrix multi-granular traffic representation and defaulted session, flow, and packet level semantic features.

variable number of features. Some of the common flow-level features are flow size (in bytes and packets), flow duration, a combination of TCP flags, as well as statistical measures (mean, min, max, standard deviation, etc.) of sizes of all packets in the flow and inter-arrival times of packets, port numbers, and transport layer protocols [27], [28], [29]. For clarity and systematic analysis, flag (e.g., ACK, SYN, FIN, PSH, URG, RST, ECE, CWR, and NS) combinations are numerically coded, which is illustrated as following:

- 1-9: Individual flags (e.g., ACK, SYN, FIN, PSH, URG, RST, ECE, CWR, NS).
- 10-14: Common combinations (SYN + ACK = 10, PSH + ACK = 11, URG + ACK = 12, FIN + ACK = 13, RST + ACK = 14).
- 15: Reserved for any uncommon or previously unseen combinations.

These features offer a balance between capturing essential characteristics and maintaining computational efficiency. Users have the flexibility to add or remove features as needed for their specific use cases.

At the session level, features provide information about the flows within. Consider a session aggregated using *src* IP address (although it applies to other aggregations as well). This includes the total number o flows and *dst* IP addresses, the unique number of *dst* IP addresses, and the total number of service ports (e.g., 10 HTTP connections, 5 DNS resolutions). Such a representation allows us to detect some of the application-level anomalies, e.g., if there are 100s of outgoing DNS requests and no user application (such as browsing) in a short window, it might indicate an infected host. Given the above definition, T-Matrix represents a session as a single data point. Since a session may consist of multiple flows, and each flow can contain multiple packets, flows and packets are represented as matrices. A session encapsulates aggregated information from its flows and is therefore represented as a single vector at the beginning of a data point.

A. T-Matrix Encoding

Next, we present the process of encoding the multi-granular semantic features extracted from traffic data into a standardized format suitable for T-Attent, the second important component of UniNet. The encoding process involves the following steps: tokenization, defining the vocabulary to represent features, and designing the final format for representing input.

1) Tokenization: Tokenization breaks down textual information into manageable units (tokens) that DL models can process and analyze [30]. All traffic features corresponding to a single data point (e.g., packet sequence) should be represented as a single token. In this way, the model provides insights into which specific features contributed to the detection of an anomaly, which not only enhances the ability to detect complex attack patterns but also improves the explainability of the results (briefly discussed in Section VI). Unlike natural languages that share common characters and tokens, network traffic features are heterogeneous and the patterns are protocol-based [31]. As shown in Figure 2, features such as direction, port number, protocol, and TCP flags are categorical, while packet length and inter-arrival time (IAT) are continuous. To unify this diverse data into a consistent format for model training, below we employ a tokenization method and define a vocabulary. Tokenization techniques [32], [33] split data into tokens. We handle categorical features by assigning each category a unique token, thereby converting data into a numerical format for processing. However, directly using continuous values can lead to poor model performance due to issues like overfitting and sensitivity to outliers [34], [35], [36]. Therefore, we use binning to improve model convergence during training.

There are three commonly used binning methods [37], [38]: equal-width, equal-frequency, and clustering. Equal-width binning creates intervals of equal size, suitable for uniformly distributed data but it is less effective with outliers; in network traffic, attacks can be outliers. Equal-frequency binning distributes data points evenly across bins, managing skewed distributions well. Clustering, using algorithms like k-means, groups data by similarity, revealing inherent structures but requires more processing time [39]. We choose equal-frequency binning in T-Matrix, for its efficiency and ability to minimize the impact of outliers.

2) Vocabulary: Vocabulary is the set of unique tokens a tokenization system utilizes during training. The design of the vocabulary must balance compression (using fewer tokens to represent more information) with model performance. While higher compression can speed up processing and extend context length, it may sacrifice the ability of models to capture fine-grained details [30]. A very small vocabulary

TABLE III
TOKEN IDS, VALUES, AND DESCRIPTIONS

Token ID	Value	Description
0	0	Token used for '0' in binary features.
1-1024	1-1024	Conventional port numbers for specific services.
1025	8080	HTTP port.
1026	3306	MySQL port.
1027	Other ports	Ports other than the specified well-known ports, and '-1' in port representations.
1028-1038	Reserved	Reserved for future ports or protocols.
1040	[MASK]	Masking purposes in representation learning.
1041	[PAD]	Padding sequences in representation learning.

size risks oversimplifying diverse data, leading to information loss and potential overfitting [40], [41]. On the other hand, a vocabulary that is too large can be computationally expensive and impractical given resource constraints [42], [43].

For categorical features, we need to decide the range of values. Port numbers are numerical identifiers used to distinguish different applications or services on a network, ranging from 0 to 65,535. However, using all 65,535 values is impractical, as it would require an immense amount of computational resources and result in large model sizes. Instead, we focus on commonly used ports that have significant meaning in traffic analysis. This includes the well-known ports from 1-1024, in addition to any custom application ports such as 8080 (HTTP) and 3306 (MySQL). Thus, we use 1024 as a base, adding specific tokens for special ports, future protocols, and other purposes. The final settings are given in Table III; the vocabulary size is 1042. This results in a total of 1042 tokens, including 2 special tokens, [MASK] and [PAD], for masked token prediction (explained later in Section IV-B1) and padding data with insufficient lengths. We bin continuous features into 1042 bins, which also function as normalization. As extreme values can impact this method, we carry out data cleaning to remove such values.

B. T-Matrix Format

Considering the need to perform various network traffic analysis tasks, the input format must be sufficiently general to handle different scenarios. Thus, the input dataset will be in the format of a dictionary containing five keys:

1) `input` represents the sequence generated in Section III-A1, containing information about the [MASK] token. The masking ratio η indicates the proportion of features in the tokenization that are masked. For example, when using the model for unsupervised learning tasks, such as anomaly detection, we set $0 < \eta \leq 1$. For supervised classification tasks, we set $\eta = 0$.
2) `true value` represents the ground truth of the masked tokens. The values are all 0 except for the masked parts. To refine the loss function, we use the negative log loss function for model training.
3) `mask index` indicates the indices of [MASK] tokens, facilitating the calculation of the loss function by identifying which parts of the input sequence are masked.
4) `segment label` separates session-level, flow-level, and packet-level features, indicating which features are at the flow level and which are at the packet level. We detect transitions between different flows by observing changes from $0 \rightarrow 1$ or $1 \rightarrow 0$ in the segment label sequences.
5) `sequence label` is used for handling supervised learning problems, providing labels for sequences to support classification and other tasks.

An example is shown in Table IV.

TABLE IV
THE ILLUSTRATION OF FINAL INPUT FORMAT. THE HIGHLIGHT PARTS REPRESENT THE MASKED TOKENS

Key	Example
input	[0,1,54,16,**1040**,**1040**,5,1,1,...]
true value	[0,0,0,0,**45**,**85**,1,1,...]
mask index	[0,0,0,0,**1**,**1**,0,0,...]
segment label	[0,0,0,0,0,**0**,**0**,**1**,**1**,...]
sequence label	[0] or [1] or [...]

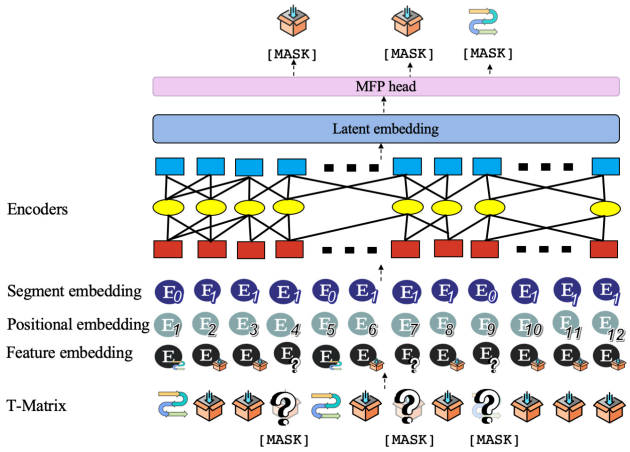


Fig. 3. The architecture of T-Attent.

IV. T-ATTENT ARCHITECTURE

T-Attent is designed to handle the heterogeneous and diverse network traffic data by generating corresponding latent embeddings. The architecture of T-Attent is shown in Figure 3. It consists of several layers that work together to process and analyze the data effectively: embedding techniques, multiple encoder layers, and a masked prediction head for latent representation learning.

A. Embedding and Encoding Layers

To effectively represent network traffic data within our attention-based model, T-Attent employs several embedding techniques. To enable the attention mechanism to effectively distinguish and integrate information across multiple granularities, we incorporate a *hierarchical segmentation embedding*. Each input token is tagged with a segment label that identifies its level of granularity—specifically, packet-, flow-, or session-level. These labels are embedded alongside positional encodings and token embeddings, allowing the model to learn cross-level dependencies in a fully data-driven manner without imposing rigid attention constraints or handcrafted guidance. For example, segment labels are assigned as follows: all packet-level tokens are labeled as [0], flow-level tokens as

[1], and session-level tokens as [2]. A combined multi-granular input sequence may thus be represented by segment labels such as [2, 1, 0, 0, ..., 1, 0, 0, ...]. Each segment label is embedded into a learnable vector of the same dimension as the token and positional embeddings. If the embedding dimension is denoted as d, a segment label sequence of shape $1 \times N$ is mapped to a segment embedding matrix $S_{\text{emb}} \in \mathbb{R}^{N \times d}$. Likewise, positional embeddings are represented as $P_{\text{emb}} \in \mathbb{R}^{N \times d}$. The final input to the encoder is computed as the element-wise sum of the token embeddings T_{emb}, segment embeddings, and positional encodings:

$$\text{Input} = T_{\text{emb}} + S_{\text{emb}} + P_{\text{emb}}.$$

This design enables the model to distinguish and attend across different granularities in a unified manner, while preserving architectural simplicity and maintaining generalizability across tasks. Additionally, the T-Attent also leverages a lightweight ViT encoder layer [44] to process inputs from T-Matrix. This encoder comprises a small number of attention heads and feed-forward layers. Unlike traditional transformers, T-Attent uses a relative segmentation embedding mechanism to split inputs into multi-granular segments, enabling it to capture local structural patterns alongside high-level dependencies. The self-attention mechanism dynamically computes weights of different parts of the input sequence, allowing the model to capture interactions between packets and flows (e.g., linking a DNS lookup to a subsequent HTTP connection). Moreover, we utilize learnable positional embeddings [45], [46] to encode the sequential order of packets within a flow, enabling the self-attention to capture essential temporal dependencies.

B. UniNet Training With Different Heads

We now present the learning phase of our framework, where UniNet is trained to generate encoded embeddings of network traffic data. These encoded embeddings can then be used as input for various ML heads or further processed for specific analysis tasks (as illustrated in Figure 1). We consider three heads for different purposes: unsupervised representation learning, anomaly detection, and classification. This framework allows UniNet to be applied in different scenarios, enhancing its practical utility.

1) MFP Head for Unsupervised Learning: For unsupervised traffic representation learning (Section V-B), we introduce a new task called Masked Feature Prediction (MFP). This technique, inspired by the pretraining of LLMs [47], involves intentionally masking certain tokens in the input data during training. The model is then trained to predict these masked tokens based on the surrounding context. For this purpose, we randomly select a percentage, denoted as η (e.g., 40%), of the features within a sequence to be masked. These selected features are replaced with the [MASK] token. The model is trained to predict the token IDs of these masked features using the provided ground truth values, used for unsupervised learning, as illustrated in Figure 3.

2) Anomaly Detection Head: The anomaly detection head is implemented as a lightweight autoencoder consisting of two fully connected layers in both the encoder and decoder. The encoder maps the latent embedding to a compressed bottleneck representation, followed by symmetric decoder layers to reconstruct the input. Formally, let z denote the latent vector output from T-Attent. The encoder compresses z as follows:

$$h = \text{ReLU}(W_1 z + b_1), \quad \tilde{z} = \text{ReLU}(W_2 h + b_2)$$

where $W_1, W_2 \in \mathbb{R}^{d \times d}$, and d is the hidden size. The decoder mirrors this structure to produce the reconstruction $\hat{z}$. We compute the mean squared error (MSE) between z and $\hat{z}$ as the reconstruction loss. Samples with losses exceeding the δ-percentile threshold (e.g., 95th percentile of benign samples) are flagged as anomalous.

3) Classification Head: The classification head is a multi-layer perceptron (MLP) composed of two fully connected layers with ReLU activation, followed by a softmax output layer. Specifically, the MLP maps the latent embedding z to a probability distribution over classes:

$$h = \text{ReLU}(W_1 z + b_1), \quad y = \text{Softmax}(W_2 h + b_2)$$

where $W_1 \in \mathbb{R}^{d \times d}$, $W_2 \in \mathbb{R}^{d \times C}$, and C is the number of output classes. Cross-entropy loss is used for optimization in supervised tasks such as attack type identification or device classification.

V. Performance Evaluations

A. Experiments Settings

We acknowledge the challenge posed by the limited availability of high-quality, open-source datasets [48]. To address this limitation, we intentionally selected three diverse datasets—CIC-IDS-2018 [49], UNSW-2018 [50], and DoQ-2024 [51]—collected by different institutions across various time periods (2018 to 2024), covering a wide range of protocols, attack types, and use cases. While CIC-IDS-2018 and UNSW-2018 were collected in controlled environments, the DoQ-2024 dataset includes real-world encrypted traffic captured during visits to live websites, based on modern Web protocols such as HTTP/3 and DNS-over-QUIC, providing realistic noise and variability.

We evaluate our approach across four tasks. For anomaly detection and attack identification, benign traffic serves as background traffic, and the goal is to detect malicious flows hidden within normal activity. For IoT device classification, regular device communications represent typical network behavior, and the task is to correctly identify the device types. For website fingerprinting, particularly in the open-world setting, traffic from many unmonitored websites serves as background traffic, and the model must recognize specific monitored websites amid this noise. All three datasets are extensive in both pcap and flow tabular formats, ensuring their suitability for our diverse tasks. Further details of each dataset are provided in the subsequent sections.

All the training and testing of our models and baselines are conducted on an Nvidia RTX 4080 16GB GPU and an Intel Core i9-13900KF processor. Due to hardware constraints, we limited the embedding size and encoder depth to lightweight configurations, which also align with our goal of maintaining

TABLE V
DEFAULT HYPERPARAMETERS

Name	Value
Vocabulary Size	1,042
Number of Encoders	2
Embedding size	10
Batch Size	32
Input length	2,000
Number of attention heads	10
Masking ratio	40%
Learning rate	10e-4 warming up 10,000 steps
Loss function	Negative Log Loss (Task 1), Cross-Entropy Loss (Tasks 2-4)

TABLE VI
BINARY CONFUSION MATRIX.TP/FP: TRUE/FALSE POSITIVE; TN/FN: TRUE/FALSE NEGATIVE

	Actual class: Y	**Actual class : not** Y
Predicted: Y	TP	FP
Predicted: not Y	FN	TN

efficiency. We experiment with masking ratios ranging from 15% to 60%, finding optimal performance at 40%. The vocabulary size for tokens is set to 1042. The model utilizes 10 heads, 10 embeddings, and 2 encoder layers. The learning rate follows a warm-up schedule, starting at 0.0001 and increasing to 0.001 over 10,000 steps. Specific settings for different heads are discussed in the corresponding sections. The default values are given in Table V for all tasks.

The commonly used metrics for network security tasks include Recall (True Positive Rate, TPR), Precision, False Positive Rate (FPR), Accuracy, and Area Under the Curve (AUC). AUC is a popular metric that counters the adverse effects of class imbalance. According to Table VI, the metrics are calculated by equations:

$$\text{Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}}, \quad \text{Precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}$$
$$\text{FPR} = \frac{\text{FP}}{\text{FP}+\text{TN}}, \quad \text{Accuracy} = \frac{\text{TP}+\text{TN}}{\text{TP}+\text{TN}+\text{FP}+\text{FN}}$$

For multi-class classification, we compute the *macro* values of these metrics independently for each class and then average them across all classes. The threat model for each task is mentioned in the corresponding sections. We now evaluate UniNet and baselines for four different security tasks—Tasks 1-4 in Table II—across the three categories of unsupervised anomaly detection, supervised classification of attacks and devices, and semi-supervised website fingerprinting.

B. Task 1: Unsupervised Anomaly Detection

Threat model: In anomaly detection (Task 1), the primary goal is to detect malicious network traffic that deviates from a learned benign profile. We assume that the training dataset, organized into session-level structures, is predominantly benign but may contain a small fraction of undiscovered attacks; however, it is not extensively poisoned by adversaries. Attackers can manipulate or inject flows, adjusting timing or header fields (e.g., IP addresses, ports) to blend into normal patterns; however, they do not control the overall training pipeline or the underlying network infrastructure.

TABLE VII
DATA DISTRIBUTION FOR ANOMALY DETECTION (TASK 1)

Category	Type	Count	Distribution (%)	Label	Ratio (%)
Training	Benign	**223,662**	-	-	-
	Benign	10,000	50	0	50
Testing	DDoS	2,000	10	1	50
	DoS	2,000	10		
	BruteForce	2,000	10		
	Bot	2,000	10		
	Infiltration	2,000	10		

Dataset: We use the CSE-CIC-IDS2018 dataset [49] for this task, and after processing the input into T-Matrix format, we use only the benign traffic to train T-Attent.[2] The evaluation focuses on five types of network attacks: DDoS, DoS, BruteForce, Botnet, and Infiltration, which are categorized as malicious during the testing phase. The distribution of training and testing data is detailed in Table VII.

Input representation: The input to UniNet is structured to facilitate unsupervised learning, organized at a session level. Sessions are composed of flows grouped by the same source or destination IP (Section III). Segment labels distinguish different levels of features and different flows within the same session. The input sequence length is set to 2,000 tokens. We input all flows and their packets in the order of arrival until the sequence reaches 2,000 tokens. Any remaining tokens are padded with `[PAD]`. Each flow is represented by 8 features, and each packet by 6 features (Section III). Thus, representing a flow-packet segment requires a length of 68 features, making space for $\approx$ 30 flows within an input sequence. Given the simpler and less informative nature of packet features compared to natural language, a higher masking ratio is justified.

Baselines: The baselines we evaluate are:

1) *Machine Learning baselines:* We consider traditional ML algorithms such as Isolation Forest, One-Class SVM, Local Outlier Factor (LOF), and K-means clustering. These models rely on statistical and distance-based methods to identify anomalies. They are particularly effective for scenarios with well-defined feature spaces, offering faster training times and lower computational requirements. They have been used commonly for network traffic analysis (e.g., see [52], [53], [54], [55]).
2) *Deep learning baselines:* We implement deep learning models used in the past for network anomaly detection, including standard autoencoders (AE) [5], variational autoencoders (VAE) [56], and LSTM-based VAEs [57]. These models are good at learning hierarchical and temporal representations from raw network traffic data. AE reconstructs input data and detects anomalies based on reconstruction loss, while VAEs introduce a probabilistic framework to model data distributions. LSTM-based VAEs capture sequential dependencies in traffic patterns, enhancing anomaly detection for time-series data.

[2]In practice, the benign class is created by removing suspicious flows using rules; yet it is assumed that small part of this class contains some malicious flows [5].

TABLE VIII
COMPARISON OF BASELINE MODELS AND UNINET FOR TASK 1

	Model	Accuracy	F1 Score	Precision	Recall	AUC	FPR
Baseline	Isolation Forest	0.5260	0.5537	0.5299	0.5760	0.5312	0.3124
	One-Class SVM	0.6412	0.6337	0.6220	0.6468	0.6490	0.2581
	LOF	0.6918	0.6719	0.6505	0.6960	0.6907	0.2893
	K-means	0.5804	0.5356	0.5831	0.5412	0.5798	0.4190
	AE	0.6204	0.6037	0.6019	0.6275	0.6212	0.2750
	VAE	0.7112	0.7156	0.6924	0.7405	0.7321	0.2645
	LSTM-VAE	0.7351	0.7357	0.7348	0.7279	0.7660	0.2336
	Average	0.6437	0.6357	0.6306	0.6511	0.6528	0.2931
UniNet +	Isolation Forest head	0.6427 ↑22.16%	0.6594 ↑19.16%	0.6487 ↑22.18%	0.6815 ↑18.06%	0.6728 ↑26.41%	0.2825 ↓9.56%
	One-Class SVM head	0.7521 ↑17.29%	0.7435 ↑17.10%	0.7306 ↑17.49%	0.7579 ↑17.20%	0.7552 ↑16.36%	0.2205 ↓14.57%
	LOF head	0.7814 ↑12.95%	0.7698 ↑14.58%	0.7612 ↑17.01%	0.7807 ↑12.16%	0.7793 ↑12.81%	0.2618 ↓9.52%
	K-means head	0.6549 ↑12.84%	0.6403 ↑19.55%	0.6621 ↑13.54%	0.6304 ↑16.45%	0.6532 ↑12.65%	0.3760 ↓10.26%
	AE head	0.7854 ↑26.60%	0.7742 ↑28.26%	0.7531 ↑25.15%	0.7967 ↑27.50%	0.7835 ↑26.10%	0.2154 ↓21.68%
	VAE head	0.8023 ↑12.84%	0.7927 ↑10.77%	0.7709 ↑11.34%	0.8163 ↑10.24%	0.8034 ↑9.73%	0.1968 ↓25.59%
	LSTM-VAE head	0.8689 ↑13.57%	0.8584 ↑13.61%	0.8497 ↑15.64%	0.8679 ↑11.58%	0.8681 ↑13.37%	0.1312 ↓43.81%
	Average	0.7597 ↑18.01%	0.7526 ↑18.49%	0.7438 ↑17.98%	0.7659 ↑17.64%	0.7637 ↑17.00%	0.2406 ↓17.90%

The primary distinction between UniNet and the baseline approaches lies in the utilization of the MFP head for embedding extraction and multi-granular representation. Specifically, UniNet employs T-Matrix and embeddings generated by the MFP head, which are subsequently processed through various anomaly detection models. In contrast, baseline approaches use single-level information, such as a sequence of packets or flows. They skip this step and apply anomaly detection techniques directly to features without encoding by the MFP head.

Orchestration of UniNet: To address these threats, we employ UniNet in a two-phase, unsupervised fashion. Firstly, the MFP head (Section IV-B1) learns representative embeddings by randomly masking up to 40% of traffic features and predicting them, enabling the model to capture robust patterns of benign behavior. Once T-Attent training is complete, the MFP head is removed, and the latent embeddings generated by the final encoder layer is utilized in the next phase. Secondly, an autoencoder-based anomaly detection head refines these embeddings, using reconstruction loss to identify deviations from the learned profile.

Analysis: We present the performance of each model, both for the baselines and for our enhanced implementation using UniNet (i.e., UniNet + different heads) in Table VIII. For UniNet, we perform unsupervised representation learning using the MFP head (Section IV-B1) with T-Attent. Subsequently, the initial traffic data is embedded into a transformed space to better capture underlying patterns and anomalies. The embeddings generated by T-Attent are then fed into different baseline models (anomaly detection heads).

As depicted in Table VIII, UniNet consistently outperforms the baseline across all key metrics — the accuracy improves by an average of 18.01%, F1-score by 18.49%, precision by 17.98%, recall by 17.64%, and AUC by 17.00%. The enhancements are even more pronounced with deep learning models; in comparison to AE, UniNet registers a maximum improvement of (approximately) 27% in accuracy, 28% in F1-score, and a reduction of about 44% in FPR. These results show that UniNet accurately detects anomalous traffic patterns while significantly reducing the false positive rates. The enhanced performance of UniNet can be attributed to the effective representation learning capabilities of the MFP head when combined with T-Attent. The embedding generated by T-Attent encompasses both sequential and statistical features, leading to a more robust and comprehensive understanding.

C. Task 2: Supervised Attack Identification

Threat model: As for attack identification (Task 2), we consider a realistic network environment where attackers launch a variety of threats, while attempting to evade detection by mimicking benign traffic patterns and manipulating both flow- and session-level characteristics. An IDS aims first to distinguish malicious from benign traffic (Task 2.1), using a coarse-grained yet efficient binary classifier to handle high volumes of data. Flows flagged as malicious are then subjected to a second, more detailed classification step (Task 2.2), which identifies the specific attack type (e.g., botnet, DDoS) using a multi-class head that requires deeper contextual analysis.

A significant challenge inherent in this environment is the scarcity of labeled instances for training. Attackers often exploit this weakness, as obtaining large numbers of labeled samples for diverse or emerging attack types is prohibitively costly and time-consuming in real-world settings. This lack of labeled data can hinder the IDS's ability to generalize to new threats or achieve high classification accuracy.

Dataset: We utilize the CSE-CIC-IDS2018 dataset [49], which is predominantly composed of benign samples, reflecting real-world class imbalances and the limited availability of labeled data for certain attack types. We focus on four types of attacks: DoS, brute force, botnet, and infiltration. In Phase 1, all attacks are aggregated into a single malicious class. Phase 2 refines this classification by distinguishing among individual attack types. To address data imbalance, additional preprocessing steps are applied. The data distribution for Task 2 is presented in Table IX.

TABLE IX
INTRUSION DETECTION DATA DISTRIBUTION (TASK 2)

Category	Type	Count	Ratio (%)	Labels		Total Ratio (%)
				Phase 1	Phase 2	
Benign	Benign	**40,000**	57.04	0	0	57.04
Attack	DoS	10,196	14.54	1	1	42.96
	BruteForce	9,523	13.58		2	
	Bot	6,359	9.07		3	
	Infiltration	4,048	5.77		4	

TABLE X
PERFORMANCE METRICS FOR ATTACK DETECTION (TASK 2)

Model Type	Accuracy	Precision	Recall	F1-Score	FPR	Inference Time (us)
CD-LSTM [59]	0.9888	0.9849	0.9946	0.9898	0.0182	4.0
GRU-tFP [23]	0.9839	0.9771	0.9937	0.9854	0.0279	1.9
UniNet	**0.9941**	**0.9978**	**0.9893**	**0.9935**	**0.0018**	**0.75**

Input format: In this task, the classification is based on a single flow. It focuses on classifying individual flows based on flow-level statistics and short-term packet patterns, which is more localized in nature. Therefore, an input is a single flow and set of packets within the flow, with a length of 2,000 tokens. This format begins with flow-level features, followed by packet-level features within the same flow. If the number of packets in a flow exceeds the maximum length of the input, it will be truncated. And if the number of packets is less than the fixed length, it will be padded with `[PAD]`. We use the default flow and packet features described in Section III. The segment labels are used to separate per-packet and flow-level features, indicating which level a particular feature belongs to.

Baselines: We compare UniNet with recent sequence models: LSTM-NoD [58] and GRU-tFP (Gated Recurrent Unit) [23]. The LSTM-NoD model utilizes two LSTM models, one trained on normal-day (N) traffic and the other on attack-day (D) traffic, to estimate the likelihood of network requests being DDoS attacks [58]. The GRU-tFP model is introduced in [23] to address different tasks, including intrusion detection, in a supervised way. GRU-tFP uses the GRU model to extract traffic features hierarchically to capture both intra-flow and inter-flow correlations. To analyze the impact of T-Matrix and T-Attent, LSTM-NoD and GRU-tFP are provided with single packet-level data sequences. In contrast, UniNet uses the T-Matrix format with flow and packet level data. We also assess the ability of each model to extract meaningful traffic patterns with limited training instances per class. We employ a lightweight hierarchical transformer architecture comprising two encoder layers with an embedding size of 10, resulting in a total of 15,000 parameters. This parameter count is significantly smaller compared to LLMs, which typically contain billions of parameters. The compact design facilitates efficient execution and simplifies implementation, making it suitable for deployment in resource-constrained environments.

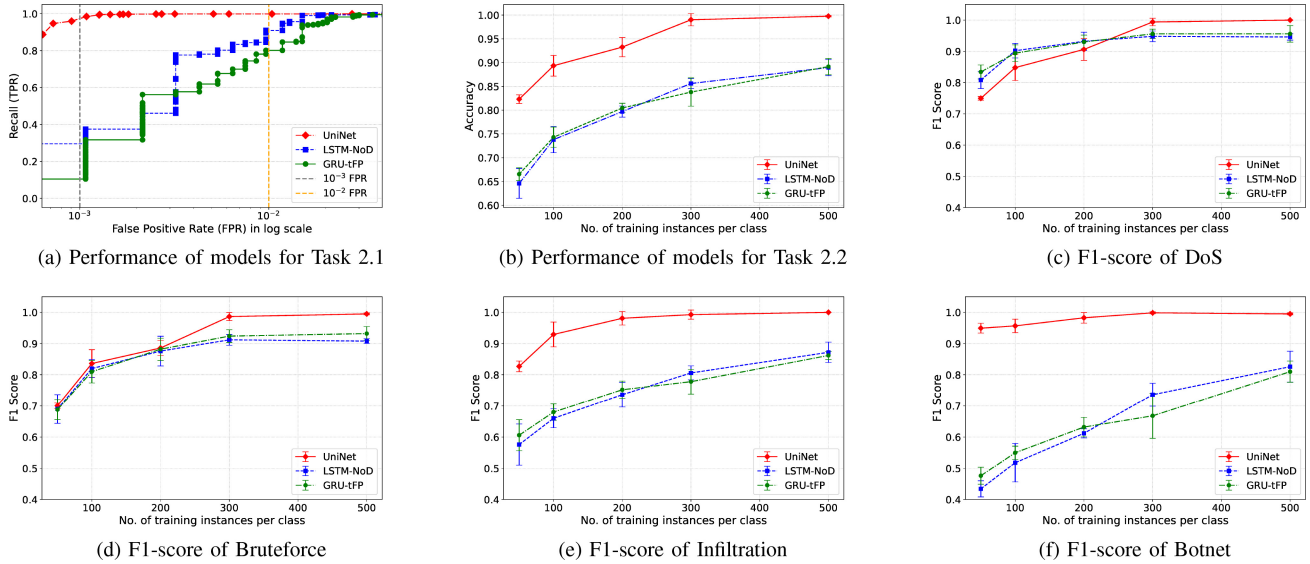
Orchestration of UniNet: While adversaries may manipulate timing and header fields to blend in with legitimate sessions, the IDS leverages session-level aggregation, flow-based features, and specialized embedding strategies to highlight anomalies that cannot be entirely concealed. Under conditions of label sparsity, we design experiments that explore the system's robustness under varying levels of labeled data availability, ranging from highly sparse (50 samples per class) to more representative distributions (500 samples per class).

Analysis: For the Phase 1 (broad detection), the results are presented in Table X. UniNet achieves the highest accuracy of 99.41% over all baselines. In the context of intrusion detection, balancing the trade-off between recall/TPR (True Positive Rate) and the False Positive Rate (FPR) is crucial. A low FPR is essential to minimize false alarms, which cost human hours for security analysis. However, this often comes at the expense of recall, due to missed detection of anomalies. Figure 4a illustrates the performance of UniNet and baseline models across different FPR values. All models achieve high recall at high FPR levels, but the real test of efficacy lies in their performance at lower FPR values. At an FPR of 10^{-2}, UniNet demonstrates an absolute increase of $\sim 14\%$ for TPR compared to the best-performing baseline (LSTM-NoD). This advantage becomes even more notable as the FPR is reduced to 10^{-3}; the TPR gap between UniNet and best performing baseline increases significantly to $\sim 68\%$. These results highlight the ability of UniNet to maintain high detection rates without sacrificing the FPR.

We test with different *training instances per class* to evaluate the information extraction capability of different models for Phase 2 (granular classification). When provided with same informative data, the model that extracts and utilizes information most effectively has a significant impact. Figure 4b gives the overall accuracy across all attacks, where UniNet exhibits an average $\sim 14\%$ accuracy improvement over the baselines. The model converges with 300 training instances per class, highlighting the effectiveness of T-Attent part in UniNet, which utilizes the self-attention mechanism to extract intrinsic patterns.

Figure 4c-4d shows the F1-scores for each attack type. Although DoS and Brute Force attacks are generally easier for all models to detect due to their prominent and distinguishable characteristics, we still see an increasing gap between UniNet and baselines with increasing training instances. As for Bot and Infiltration attacks, UniNet demonstrates a significant improvement over LSTM-NoD and GRU-tFP, particularly with a low number of training instances (e.g., 100) per class. Notably, there is an absolute increase in F1-score by $\sim 25\%$ for Infiltration and $\sim 43\%$ for Botnet compared to the best-performing baseline (GRU-tFP). This can be attributed to the limitations of LSTM-NoD and GRU-tFP in capturing long-distance dependencies, especially when features are flattened, weakening the relationship between nearby tokens. In contrast, UniNet performs well in understanding long sequences, which is important for identifying both Bot and Infiltration. These attacks often exhibit subtle, long-range dependencies in their behavior patterns that simpler models struggle to capture.

Inference time: We evaluate the inference time for the different models. LSTM-NoD model exhibits the highest inference time of 4.0 µs, whereas UniNet processing sequences in parallel, achieves the lowest inference time of 0.75 µs (see Table X).



(a) Performance of models for Task 2.1 (b) Performance of models for Task 2.2 (c) F1-score of DoS

(d) F1-score of Bruteforce (e) F1-score of Infiltration (f) F1-score of Botnet

Fig. 4. F1-scores and performance metrics for various attack types and phases in Task 2.

D. Task 3: Multi-Class Device Classification

Threat model: In IoT device classification (Task 3), the goal of the system, in this case a network defense system, is to identify the types of devices connected to the network by continuously monitoring its traffic flows, such as those in an enterprise environment. This helps the enterprise maintain awareness of all devices on its network and take action against unauthorized or rogue devices.

Dataset: We utilize the UNSW 2018 dataset [50], which encompasses a diverse array of device types (28 devices) exhibiting heterogeneous traffic patterns. To mitigate skewed data distributions, we train a multi-class classification head on a balanced subset of 15 selected device categories from the original 28, leveraging cross-entropy loss to enhance classification boundaries. Considering the dataset does not have labels, we group the traffic by MAC address based on the device name list. Since the dataset is imbalanced, we remove devices with very few data points and select 15 devices with more than 10,000 data points. For devices with an excessive number of data points, we randomly select 60,000 data points for each device type.

Input representation: In this session-level task, the data is represented as sessions, where packets grouped by a src (dst) IP address within a static time-window form a session (refer Section III). Each session may contain multiple flows; and a single flow may span multiple sessions, thereby becoming incomplete in session(s) due to the time-window splits. The data is then segmented them into sequences of 2,000 tokens based on their arrival time. The segment labels for UniNet are '0's for incomplete flow-level features and '1's for per-packet features. As for UniNet w/o T-Matrix, the segment labels are set to all '1's. Positional information is based on the arrival time of each packet. We use only the six default packet features mentioned in Section III: source/destination port representation, direction, packet size, transport layer protocol, and IAT.

Baselines: We compare UniNet with two recent sequence models for IoT fingerprinting: SANE [14] and BiLSTM-iFP [24]. The SANE model employs a similar architecture to UniNet, utilizing an attention-based structure but relying solely on per-packet features for IoT fingerprinting. Moreover, each packet is treated as a token in SANE; while in UniNet, each feature is treated as a token. The BiLSTM-iFP model extracts packet-level features and uses an enhanced bidirectional LSTM to perform device classification.

Both baseline models were implemented using single-level representations. Additionally, to analyze the impact of T-Matrix, we conduct an ablation study comparing the performance of UniNet with and without T-Matrix (UniNet-w/o-T-Matrix). Moreover, given the imbalanced in the dataset, there is a risk that classes with fewer data points, i.e., *minority classes*, may be overlooked or underrepresented in model training. To assess this, we specifically study the performance of four classes with the least number of data points: i) Android Phone, ii) Light Bulbs LiFX Smart Bulb, iii) Smart Baby Monitor, and iv) Aura Smart Sleep Sensor. A good performance on these classes would indicate that the model is not biased towards classes with larger data representation, thereby ensuring a more robust system.

Orchestration of UniNet: Our framework addresses the challenge of incomplete flows by aggregating traffic at the session level. This preserves essential contextual relationships, enabling the detection of inconsistencies in traffic behavior that may indicate adversarial manipulation.

Analysis: We focus on the performance of different methods on *minority classes*, presented in Figure 5. UniNet achieves the best performance across all metrics, with an improvement of $\sim$7% in accuracy, $\sim$8% in F1-score, and $\sim$6% in precision compared with BiLSTM-iFP. We carry out further analyses. i) To evaluate the advantages of the T-Matrix, we conduct an ablation study comparing UniNet with and without

TABLE XI
PERFORMANCE METRICS COMPARISON ACROSS OVERALL DATA AND MINORITY CLASSES. "UNINET W/O T-MATRIX" REFERS TO UNINET WITHOUT T-MATRIX, USING A SINGLE-LEVEL REPRESENTATION AS BASELINES FOR TASK 4

Methods	Overall Performance				Minority Classes Performance				Inference Time (µs)
	Accuracy	Macro-Precision	Macro-Recall	Macro-F1-Score	Accuracy	Macro-F1-Score	Recall	Precision	
SANE [14]	0.9841	0.9720	0.9830	0.9775	0.9007	0.9104	0.9302	0.8914	**0.72**
BiLSTM-iFP [24]	0.9752	0.9514	0.9598	0.9556	0.8657	0.8641	0.8538	0.8746	5.90
UniNet w/o T-Matrix (session only)	0.6213	0.5897	0.6114	0.5789	0.5721	0.4836	0.4712	0.4893	0.65
UniNet w/o T-Matrix (flow only)	0.8125	0.8050	0.7820	0.7934	0.7581	0.7649	0.7721	0.7583	0.71
UniNet w/o T-Matrix (packet only)	0.9856	0.9774	0.9811	0.9792	0.9178	0.9196	0.9402	0.8999	0.83
UniNet	**0.9901**	**0.9886**	**0.9855**	**0.9871**	**0.9398**	**0.9400**	**0.9438**	**0.9363**	0.85

the multi-level representation. The versions of *UniNet-w/o-T-Matrix* are divided into three categories, each using only one level of input: session-level, flow-level, or packet-level features. To ensure a fair comparison, session-level and flow-level features are positioned at the start of the input sequence and padded to a fixed length, limiting the number of flows that can be represented. As shown in Figure 5 and Table XI, UniNet consistently outperforms all single-level input variants by a substantial margin. Compared to the session-only model, UniNet improves overall accuracy and minority-class accuracy by ∼60%, and macro-F1 score by ∼90%. When compared to the flow-only model, UniNet achieves a relative improvement of ∼20% in overall accuracy, minority-class accuracy, and macro-F1. Even against the strong packet-only baseline, UniNet delivers additional gains—improving, minority-class accuracy by 2.4%, and macro-F1 by 2.2%. These results highlight that while packet-level features are crucial, the multi-granularity integration through T-Matrix leads to significantly more robust and generalizable performance, particularly for underrepresented classes. ii) As for the effectiveness of T-Attent, we compare the performance between *UniNet-w/o-T-Matrix (packet only)* and SANE. Both models use advanced attention-based architectures and single-level representations. The key difference lies in their tokenization mechanisms: *UniNet-w/o-T-Matrix* takes a feature as a token, whereas SANE is based on per-packet tokens. We observe a modest improvement in accuracy. By analyzing interactions between flows and packets within a session, and combining flow-level and packet-level features, UniNet generates robust device identification. This makes it significantly harder for adversaries to impersonate a targeted device class or maintain consistent false signals across multiple flows. The overall performance of different methods of device classification is summarized in Table XI.

Inference time: Table XI also provides the inference time for the different models. While BiLSTM-iFP takes 5.9 µs, UniNet, with an inference time of 0.85 µs, is significantly faster, making it a better candidate for deployments.

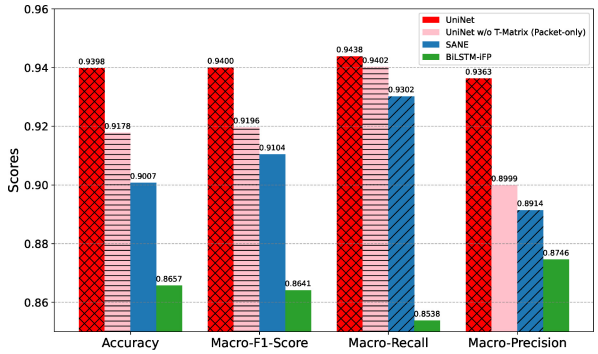


Fig. 5. Performance comparison of *minority classes* for Task 4.

E. Task 4: Encrypted Website Fingerprinting

Threat model: In website fingerprinting (Task 4), an adversary aims to infer which website a user is visiting based on observed traffic patterns, even when packet payloads are encrypted. We assume the attacker has a vantage point to observe client communication (e.g., compromised router) and sufficient knowledge to inspect flow and session-level characteristics, particularly in HTTP/3 (QUIC) and DNS-over-QUIC (DoQ) traffic. In the *closed-world* setting, the user activities are restricted to a known, "monitored" set of websites, each of which the attacker has previously profiled through multiple training samples. Here, the adversary's objective is to classify which monitored site the user is visiting. In the *open-world* setting, the users also visit an extensive set of "unmonitored" sites. The attacker thus seeks to determine whether a given visit is to one of the monitored sites, or to an unmonitored one, despite incomplete knowledge of these unknown destinations.

Dataset: We use the recent DoQ-2024 [51], which captures network traffic from HTTP/3 and DoQ Web sessions across four vantage points. The dataset includes over 75,000 unique websites, with 500 monitored QUIC sites visited 1,280 times each, and additional unmonitored sites visited 4 times each.

Input representation: This session-based collection allows us to extract aggregated session-level features, including the total number of flows, average and standard deviation of flow sizes and durations, total inbound and outbound bytes, and the inbound/outbound traffic ratio. These eight session-level features are concatenated with 1,992 packet-level features to form a 2,000-dimensional input vector. In our UniNet architecture, we incorporate a relative embedding to distinguish session-level from packet-level segments, ensuring effective attention across both granularity.

Baselines: We evaluate our method against several baselines, including models introduced in related works. Specifically, we compare our approach to an AutoWFP model [20], the TMWF model [21], and TDoQ model [22]. AutoWFP is based on LSTM. Although TMWF and TDoQ are based on transformer, their architectures differ significantly.

TABLE XII
PERFORMANCE OF CLOSED-WORLD SETTING (300 CLASSES)

Method	Accuracy (%)	Macro-Precision (%)	Macro-F1 Score (%)
AutoWFP	91.1	89.5	89.8
TMWF	92.9	91.0	91.5
TDoQ	96.8	95.0	95.7
UniNet	**98.9**	**98.3**	**98.6**

TMWF employs a traditional transformer by Vaswani et al. [9], while TDoQ model utilizes a ViT-based patch embedding design [44]. UniNet further distinguishes itself by incorporating a multi-granularity representation, T-Matrix, combining session-level features with packet-level details, along with an expanded and more sophisticated encoding strategy (refer Section III-A).

Orchestration of UniNet: QUIC/DoQ encryption conceals packet payloads, but does not entirely mask metadata such as flow sizes, inter-arrival times, and directionality, enabling the attacker to extract session-level aggregates (e.g., total flows) and packet-level features for fingerprinting. By constructing a robust signature from these features, the attacker attempts to discriminate among thousands of potential websites in both closed-world and open-world environments.

Analysis: In our *closed-world* experiments involving 300 monitored websites, we evaluate four fingerprinting methods using metrics such as accuracy, macro-precision, and F1 score. As shown in Table XII, UniNet achieves an accuracy of 98.9%, representing an absolute improvement of approximately 2% over the next best method, TDoQ (96.8%). Furthermore, UniNet enhances macro-precision and F1 score by approximately 3% each compared to TDoQ. These substantial improvements demonstrate that the multi-granular transformer architecture of UniNet significantly outperforms baseline methods, thereby establishing a new benchmark in closed-world website fingerprinting.

Open-world website fingerprinting: To evaluate UniNet's performance in a realistic *open-world* scenario, we consider the top 100 QUIC-enabled domains, each generating 360 traces (36,000 traces in total) as "monitored", and assigned them to 100 distinct classes. An unmonitored class comprised 45,000 other websites, each contributing four traces, resulting in 180,000 traces. Importantly, no unmonitored website appears in both the training and test sets. As per [51], traces were randomly collected from various locations to ensure diversity. We employ a 75:25 train–test split for the monitored classes and a balanced 1:1 split for the unmonitored class. This features a highly imbalanced testing ratio of approximately **1:10** between monitored and unmonitored traces. We assess the TPR against the FPR in detecting monitored sites. As is common in literature (e.g., [20], [21]), we adopt a binary setting by aggregating all monitored classes into a single positive category and all unmonitored classes into a single negative category.

As depicted in Figure 6, UniNet achieves a higher TPR at low FPR levels compared to baseline methods, demonstrating superior discriminative capabilities between monitored and unmonitored traffic. Notably, UniNet attains a TPR of 81% at a low FPR of 10^{-3}, surpassing TDoQ (58%), TMWF (49%), and AutoWFP (35%). High TPRs at low FPRs indicate that UniNet can accurately identify monitored websites while maintaining a low rate of misclassification for unmonitored websites.

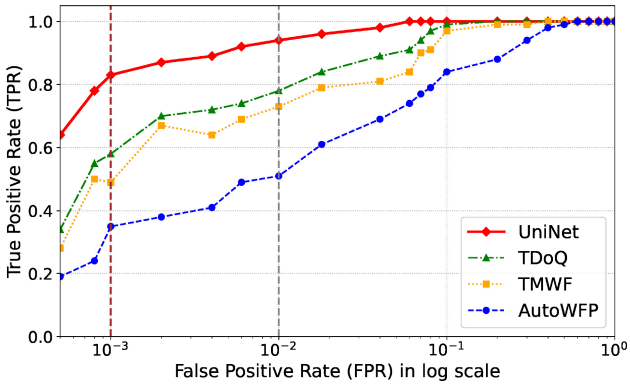


Fig. 6. Performance of open-world website fingerprinting.

Inference time: UniNet achieves the lowest average inference time of 0.15 µs, close to that of TDoQ (0.16 µs) and approximately one-third of TMWF's (0.45 µs), while being just $\approx 3\%$ of AutoWFP (4.83 µs).

VI. DISCUSSIONS AND FUTURE WORKS

We now discuss the practical considerations regarding the implementation and deployment of UniNet.

Model complexity and running time: For most tasks (Task 2-4), we utilize a lightweight hierarchical transformer architecture, achieving a training time of approximately 30 seconds per epoch with a batch size of 64 samples. This demonstrates the efficiency of training. The inference time analysis shows that UniNet achieves shorter inference time compared to DL baselines. For Task 1, which focuses on representation learning for traffic understanding, the model requires more data and time to train. However, this investment benefits deployment, as the pre-trained representation accelerates convergence in downstream models, ensuring overall efficiency in practical applications.

False alarm rate: We emphasize the importance of controlling false alarms, as real-world deployment necessitates low false positive rates to reduce the operational burden on network administrators. Through our evaluations of FPR vs. TPR across multiple tasks, we demonstrate the effectiveness of UniNet in maintaining a low false positive rate, making it a practical and reliable choice for network security applications.

Looking ahead, there are opportunities to enhance the architecture and expand its capabilities.

Explainable AI (XAI) solutions: While UniNet excels in extracting contextual relationships through its attention mechanisms, its reliance on these techniques poses interpretability challenges. As a next step, we plan to incorporate XAI solutions, such as attention visualization and feature attribution, to enhance transparency and enable analysts to validate decisions. However, current XAI techniques for transformer-based models, such as gradient-based [60], attention-score-based [61], or hybrid methods [62], are still in the early stages of

development and yet to be adopted. This gap presents an ongoing challenge that we are actively exploring.

Robustness against generative evasion attacks: We will evaluate the robustness of our UniNet against adversarial attacks. In particular, we assess its resilience to evasion techniques—a critical issue in traffic analysis. Attackers may use methods such as traffic manipulation, adversarial perturbations, or obfuscation to circumvent machine learning-based traffic analysis systems.

VII. Related Works

Below, we discuss three critical stages in the ML-based traffic analysis pipeline: feature representation, feature encoding, and model development. By examining current approaches at each stage, we identify trade-offs that underscore the need for a unified, more adaptive framework.

A. Feature Representation

Existing feature representation techniques fall mainly into two categories: *bit-level* and *semantic* representations. *Bit-level* representation uses the raw binary bits from the packet header to represent each packet [63], [64], [65]. This method can be enhanced to ensure field alignment between packets of different protocols, e.g., using padding [63]. Since the header info of each packet is encoded using bit values, this is a per-packet representation. However, such a simple encoding has two serious limitations:

- Bit-level representation of header hard codes certain fields, such as src/dst IP address, leading to model overfitting. For example, in most cases, a benign computer that is infected or breached may start communicating with a C&C server. However, if the model has seen only benign traffic from this IP address, then it would likely classify the attack flow as malicious because of overfitting the IP address. nPrint proposed in [63] exhibits this overfitting tendency as the results are dependent on attacker IP addresses. Similarly, due to randomness, encoding ephemeral ports as such is not useful and might mislead a model.
- When using bit-level features for unsupervised representation learning, the smallest token unit is typically one byte (e.g., as in [66]). This approach can disrupt meaningful fields due to the varying field sizes. For example, the 16-bit port number in the header would be split into two tokens instead of being represented by a single token. Furthermore, bit-level representation increases the model size when provided as input to a sequence model, leading to a higher consumption of resources (compute and memory), besides increasing the inference time. For instance, a header with a minimum of 20 bytes would require at least 20 tokens to represent a single packet.

Semantic representations typically aggregate multiple packets or flows into constant-size feature vectors. For instance, repeated failed connection attempts to diverse destinations can signify bot activity reaching out to command-and-control (C&C) servers. Aggregated features are widely used in network security tasks, such as anomaly/attack detection [26], botnet detection [11], fingerprinting [21], etc. While semantic features can capture meaningful higher-level indicators (e.g., port usage, flow durations), they rely heavily on domain expertise. This makes them less flexible in scenarios with limited or evolving domain knowledge.

B. Feature Encoding for ML Training

Feature encoding transforms network traffic data into numeric representations suitable for ML models [11]. The process begins with normalizing heterogeneous data into a unified format, ensuring consistency and facilitating effective encoding. After normalization, data is tokenized into its minimum units for fine-grained analysis. These tokens are then embedded to extract relationships essential for understanding network behaviors. However, existing encoding methods often fall short in practical network traffic analysis [66], [67]. For instance, one-hot encoding, commonly used for categorical features like port numbers, creates high-dimensional sparse vectors, thereby increasing the computational complexity and the risk of overfitting [11]. Embedding techniques like Word2Vec [68] have been adopted in NLP, with newer contextual embedding methods proving more effective [69]. However, current approaches often use raw hex numbers for tokens [66], [67], which fragment fields into less meaningful pieces. Treating entire packets as single tokens has been proposed but poses challenges due to high dimensionality, leading to large vocabularies that complicate training [31]. Additionally, most of these works overlook sequential information between packets, such as inter-arrival time (IAT), which is helpful in capturing temporal patterns in network traffic.

C. Models for Network Traffic Analysis

A wide range of models have been developed for analyzing network traffic. In [70] and the works it surveys, statistical methods, ML, and DL models have been widely applied to tasks such as anomaly detection, device and website fingerprinting, location inference, quality of experience (QoE) measurement, and traffic classification. Statistical models rely on well-established statistical principles to identify anomalies or deviations from normal traffic patterns [71], [72], [73]. However, they often struggle with complex, evolving threats, as they rely on predefined statistical assumptions that attackers can circumvent. ML models offer greater flexibility by being able to learn from data. Techniques such as decision trees, support vector machines (SVM), and ensemble methods like random forests have been widely used to classify network traffic and detect intrusions [52], [53], [54], [55], [74]. These models can adapt to new data, improving detection rates over time. However, they often require significant feature engineering and may struggle with the high dimensionality of network data. DL models, including CNNs, recurrent neural networks (RNNs), and transformers, are capable of extracting meaningful information from raw data, capturing sequential patterns and relationships that traditional ML models might miss [23], [27]. DL models are also particularly good at handling large-scale data and can potentially adapt to various

types of threats and attacks. Nevertheless, they require substantial computational resources and large labeled datasets for training and model maintenance, which can be a barrier to their widespread adoption. A common disadvantage of the current solutions is that they often rely on task-specific models, which may not generalize well across different types of network anomalies or attack vectors.

VIII. Conclusion

In this work, we presented UniNet, a unified framework for network traffic analysis that introduces the T-Matrix multi-granularity representation and the lightweight attention-based model, T-Attent. UniNet addresses key limitations of existing approaches by seamlessly integrating session-level, flow-level, and packet-level features, enabling comprehensive contextual understanding of network behavior. Its adaptable architecture, featuring task-specific heads, supports a variety of network security tasks, including anomaly detection, attack classification, IoT device fingerprinting, and encrypted website fingerprinting. Extensive evaluations across diverse datasets demonstrated the superiority of UniNet over state-of-the-art methods in terms of accuracy, false positive rates, scalability, and computational efficiency.

References

[1] "Google transparency report." Google. 2024. [Online]. Available: https://transparencyreport.google.com/https/overview

[2] B. Anderson and D. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proc. SIGKDD*, 2017, pp. 1723–1732.

[3] D. M. Divakaran, K. W. Fok, I. Nevat, and V. L. Thing, "Evidence gathering for network security and forensics," *Digit. Investigat.*, vol. 20, pp. S56–S65, Mar. 2017.

[4] I. Nevat et al., "Anomaly detection and attribution in networks with temporally correlated traffic," *IEEE/ACM Trans. Netw.*, vol. 26, no. 1, pp. 131–144, Feb. 2018.

[5] Q. P. Nguyen, K. W. Lim, D. M. Divakaran, K. H. Low, and M. C. Chan, "GEE: A gradient-based explainable variational autoencoder for network anomaly detection," in *Proc. EEE CNS*, 2019, pp. 91–99.

[6] P. Bosshart et al., "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, pp. 87–95, 2014.

[7] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2021, pp. 3431–3446.

[8] G. Zhou, Z. Liu, C. Fu, Q. Li, and K. Xu, "An efficient design of intelligent network data plane," in *Proc. 32nd USENIX Security Symp. (USENIX Security)*, 2023, pp. 6203–6220.

[9] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, 2017, pp. 1–15.

[10] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. 25th Annu. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2018.

[11] S. T. Jan et al., "Throwing darts in the dark? Detecting bots with limited data using neural data augmentation," in *Proc. IEEE SP*, 2020, pp. 1190–1206.

[12] Y. Yin, Z. Lin, M. Jin, G. Fanti, and V. Sekar, "Practical GAN-based synthetic ip header trace generation using netshare," in *Proc. ACM SIGCOMM Conf.*, 2022, pp. 458–472.

[13] M. Shen, K. Ji, Z. Gao, Q. Li, L. Zhu, and K. Xu, "Subverting Website fingerprinting defenses with robust traffic representation," in *Proc. 32nd USENIX Security Symp. (USENIX Security)*, 2023, pp. 607–624.

[14] B. Wu, P. Gysel, D. M. Divakaran, and M. Gurusamy, "ZEST: Attention-based zero-shot learning for unseen IoT device classification," in *Proc. IEEE Netw. Operations Manage. Symp. (NOMS)*, 2024, pp. 1–9.

[15] X. Jiang et al., "NetDiffusion: Network data augmentation through protocol-constrained traffic generation," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 8, no. 1, pp. 1–32, 2024.

[16] B. Claise, B. Trammell, and P. Aitken, "Specification of the IP flow information export (IPFIX) protocol for the exchange of flow information," IETF, RFC 7011, 2013. [Online]. Available: http://www.rfc-editor.org/rfc/rfc7011.txt

[17] L. Bilge, D. Balzarotti, W. Robertson, E. Kirda, and C. Kruegel, "Disclosure: Detecting botnet command and control servers through large-scale NetFlow analysis," in *Proc. ACSAC*, 2012, pp. 129–138.

[18] D. Brauckhoff, X. Dimitropoulos, A. Wagner, and K. Salamatian, "Anomaly extraction in backbone networks using association rules," *IEEE/ACM Trans. Netw.*, vol. 20, no. 6, pp. 1788–1799, Dec. 2012.

[19] Z. Liu et al., "Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric DDoS attacks with programmable switches," in *Proc. 30th USENIX Security Symp. (USENIX Security)*, 2021, pp. 3829–3846.

[20] V. Rimmer, D. Preuveneers, M. Juarez, T. van Goethem, and W. Joosen, "Automated Website fingerprinting through deep learning," in *Proc. 25th Annu. Netw. Distrib. Syst. Security Symp.*, San Diego, CA, USA, 2018, pp. 1–15.

[21] Z. Jin, T. Lu, S. Luo, and J. Shang, "Transformer-based model for multi-tab Website fingerprinting attack," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2023, pp. 1050–1064.

[22] L. Csikor, Z. Lian, H. Zhang, N. Lakshmanan, and D. M. Divakaran, "DNS-over-QUIC and HTTP/3 in the era of transformers: The new Internet privacy battle," *IEEE Commun. Mag.*, early access, Jun. 2, 2025, doi: 10.1109/MCOM.004.2400680.

[23] J. Qu et al., "An input-agnostic hierarchical deep learning framework for traffic fingerprinting," in *Proc. 32nd USENIX Security Symp. (USENIX Security)*, 2023, pp. 589–606.

[24] S. Dong, Z. Li, D. Tang, J. Chen, M. Sun, and K. Zhang, "Your smart home can't keep a secret: Towards automated fingerprinting of IoT traffic," in *Proc. 15th ACM Asia Conf. Comput. Commun. Security (AisaCCS)*, 2020, pp. 47–59.

[25] X. Jiang, H.-R. Zhang, and Y. Zhou, "Multi-granularity abnormal traffic detection based on multi-instance learning," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 1467–1477, Apr. 2024.

[26] A. Alsaheel et al., "ATLAS: A sequence-based learning approach for attack investigation," in *Proc. 30th USENIX Security Symp. (USENIX security)*, 2021, pp. 3005–3022.

[27] T. Van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, vol. 27, 2020, pp. 1–18.

[28] M. Piskozub, F. De Gaspari, F. Barr-Smith, L. Mancini, and I. Martinovic, "Malphase: Fine-grained malware detection using network flow data," in *Proc. ACM Asia Conf. Comput. Commun. Security*, 2021, pp. 774–786.

[29] D. Barradas, N. Santos, L. Rodrigues, S. Signorello, F. M. Ramos, and A. Madeira, "FlowLens: Enabling efficient flow classification for ML-based network security applications," in *Proc. NDSS*, 2021, pp. 1–18.

[30] T. Limisiewicz, J. Balhar, and D. Mareček, "Tokenization impacts multilingual language modeling: Assessing vocabulary allocation and overlap across languages," in *Findings of Assoc. Comput. Linguist. (ACL)*, 2023, pp. 1–19.

[31] F. Le, M. Srivatsa, R. Ganti, and V. Sekar, "Rethinking data-driven networking with foundation models: Challenges and opportunities," in *Proc. 21st ACM Workshop Hot Topics Netw.*, 2022, pp. 188–197.

[32] S. Yehezkel and Y. Pinter, "Incorporating context into subword vocabularies," in *Proc. 17th Conf. Eur. Chapter Assoc. Comput. Linguist.*, 2023, pp. 623–635.

[33] K. Gurugubelli, S. Mohamed, and K. S. R. Krishna, "Comparative study of Tokenization algorithms for end-to-end open vocabulary keyword detection," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2024, pp. 12431–12435.

[34] M. Sugiyama and K. M. Borgwardt, "Finding statistically significant interactions between continuous features," in *Proc. IJCAI*, 2019, pp. 3490–3498.

[35] Y. Chen, S. Liu, and X. Wang, "Learning continuous image representation with local implicit image function," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 8628–8638.

[36] A. Asudeh, N. Shahbazi, Z. Jin, and H. Jagadish, "Identifying insufficient data coverage for ordinal continuous-valued attributes," in *Proc. Int. Conf. Manage. Data*, 2021, pp. 129–141.

[37] J. Dougherty, R. Kohavi, and M. Sahami, "Supervised and unsupervised discretization of continuous features," in *Proc. Mach. Learn.*, 1995, pp. 194–202.

[38] Y. Gorishniy, I. Rubachev, and A. Babenko, "On embeddings for numerical features in tabular deep learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24991–25004.

[39] M. G. Omran, A. P. Engelbrecht, and A. Salman, "An overview of clustering methods," *Intell. Data Anal.*, vol. 11, no. 6, pp. 583–605, 2007.
[40] W. Chen, Y. Su, Y. Shen, Z. Chen, X. Yan, and W. Wang, "How large a vocabulary does text classification need? A variational approach to vocabulary selection," in *Proc. NAACL-HLT*, 2019, pp. 3487–3497.
[41] C. Toraman, E. H. Yilmaz, F. Şahinuç, and O. Ozcelik, "Impact of tokenization on language models: An analysis for turkish," *ACM Trans. Asian Low-Resource Lang. Inf. Process.*, vol. 22, no. 4, pp. 1–21, 2023.
[42] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. R. Salakhutdinov, and Q. V. Le, "XLNet: Generalized autoregressive pretraining for language understanding," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, p. 517.
[43] H. Touvron et al., "LLAMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
[44] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. 9th Int. Conf. Learn. Represent.*, 2021, pp. 1–22.
[45] K. Wu, H. Peng, M. Chen, J. Fu, and H. Chao, "Rethinking and improving relative position encoding for vision transformer," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 10033–10041.
[46] Z. Huang, D. Liang, P. Xu, and B. Xiang, "Improve transformer models with better relative position embeddings," 2020, *arXiv:2009.13658*.
[47] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
[48] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark NIDS datasets," in *Proc. IEEE 9th Eur. Symp. Security Privacy (EuroSP)*, 2024, pp. 658–675.
[49] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSP*, vol. 1, 2018, pp. 108–116.
[50] A. Sivanathan et al., "Classifying IoT devices in smart environments using network traffic characteristics," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1745–1759, Aug. 2019.
[51] L. Csikor, "DoQ+QUIC Web traffic dataset," IEEE Dataport, 2024. [Online]. Available: https://dx.doi.org/10.21227/km5h-g294
[52] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Min.*, 2008, pp. 413–422.
[53] K.-L. Li, H.-K. Huang, S.-F. Tian, and W. Xu, "Improving one-class SVM for anomaly detection," in *Proc. Int. Conf. Mach. Learn. Cybern.*, vol. 5, 2003, pp. 3077–3081.
[54] Z. Xu, D. Kakde, and A. Chaudhuri, "Automatic hyperparameter tuning method for local outlier factor, with applications to anomaly detection," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, 2019, pp. 4201–4207.
[55] G. Münz, S. Li, and G. Carle, "Traffic anomaly detection using K-means clustering," in *Proc. GI/ITG Workshop MMBNET*, vol. 7, 2007, pp. 1–8.
[56] J. Pereira and M. Silveira, "Unsupervised anomaly detection in energy time series data using variational recurrent autoencoders with attention," in *Proc. IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, 2018, pp. 1275–1282.
[57] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.
[58] W. J.-W. Tann, J. J. W. Tan, J. Purba, and E.-C. Chang, "Filtering DDoS attacks from unlabeled network traffic data using online deep learning," in *Proc. ACM Asia Conf. Comput. Commun. Security (AsiaCCS)*, 2021, pp. 432–446.
[59] A. Lazaris and V. K. Prasanna, "An LSTM framework for modeling network traffic," in *Proc. IFIP/IEEE Symp. Integr. Netw. Service Manage. (IM)*, 2019, pp. 19–24.
[60] A. Ali, T. Schnake, O. Eberle, G. Montavon, K.-R. Müller, and L. Wolf, "XAI for transformers: Better explanations through conservative propagation," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 435–451.
[61] S. Abnar and W. Zuidema, "Quantifying attention flow in transformers," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguist.*, 2020.
[62] H. Chefer, S. Gur, and L. Wolf, "Transformer interpretability beyond attention visualization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 782–791.
[63] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, "New directions in automated traffic analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2021, pp. 3366–3383.
[64] X. Meng, Y. Wang, R. Ma, H. Luo, X. Li, and Y. Zhang, "Packet representation learning for traffic classification," in *Proc. 28th ACM SIGKDD Conf. Knowl. Disc. Data Min.*, 2022, pp. 3546–3554.
[65] M. Swarnkar and N. Sharma, "OptiClass: An optimized classifier for application layer protocols using bit level signatures," *ACM Trans. Privacy Security*, vol. 27, no. 1, pp. 1–23, 2024.
[66] X. Meng, C. Lin, Y. Wang, and Y. Zhang, "NetGPT: Generative pretrained transformer for network traffic," 2023, *arXiv:2304.09513*.
[67] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.
[68] T. Mikolov et al., "Distributed representations of words and phrases and their compositionality," in *Proc. NIPS*, 2013, pp. 3111–3119.
[69] Y. Liu et al., "RoBERTa: A robustly optimized bert pretraining approach," 2019, *arXiv:1907.11692*.
[70] Y. Feng et al., "Unmasking the Internet: A survey of fine-grained network traffic analysis," *IEEE Commun. Surveys Tuts.*, early access, Feb. 25, 2025, doi: 10.1109/COMST.2025.3545541.
[71] S. Fernandes, R. Antonello, T. Lacerda, A. Santos, D. Sadok, and T. Westholm, "Slimming down deep packet inspection systems," in *Proc. IEEE INFOCOM Workshops*, 2009, pp. 1–6.
[72] X. Wang, J. Jiang, Y. Tang, B. Liu, and X. Wang, "StriD^2FA: Scalable regular expression matching for deep packet inspection," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2011, pp. 1–5.
[73] F. Simmross-Wattenberg, J. I. Asensio-Perez, P. Casaseca-de-la Higuera, M. Martin-Fernandez, I. A. Dimitriadis, and C. Alberola-Lopez, "Anomaly detection in network traffic based on statistical inference and α-stable modeling," *IEEE Trans. Dependable Secur. Comput.*, vol. 8, no. 4, pp. 494–509, Jul./Aug. 2011.
[74] Y. Feng, J. Li, D. Sisodia, and P. Reiher, "On explainable and adaptable detection of distributed denial-of-service traffic," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 2211–2226, Jul./Aug. 2024.

Binghui Wu (Graduate Student Member, IEEE) received the B.Eng. degree from The Chinese University of Hong Kong, Shenzhen, and the M.Eng. degree from the National University of Singapore, where he is currently pursuing the Ph.D. degree with the Department of Electrical and Computer Engineering. His research interests include network traffic analysis, deep learning, and AI for network security.

Dinil Mon Divakaran (Senior Member, IEEE) received the Doctoral degree from the joint lab of INRIA and Bell Labs, ENS Lyon, France. He is a Senior Principal Scientist and a Group Leader (Network and System Security) with the Institute for Infocomm Research, A*STAR, Singapore. He is also an Adjunct Assistant Professor with the School of Computing, National University of Singapore. His research interests are network security, web security, and AI security.

Gurusamy Mohan (Senior Member, IEEE) received the Ph.D. degree in computer science and engineering from the Indian Institute of Technology Madras in 2000. He joined the National University of Singapore (NUS) in June 2000, where he is currently an Associate Professor with the Department of Electrical and Computer Engineering (ECE). He is currently serving as an Associate Head of Graduate Programmes with the ECE Department and also as an Assistant Dean of Graduate Programmes with the NUS College of Design and Engineering. He has over 270 publications to his credit including two books and three book chapters in the area of optical networks. His research experience and interests are in the areas of Internet of Things, 5G, software defined networks, metaverse, cloud, and optical networks. He is serving on the editorial board for *Elsevier Computer Networks* journal and *Springer Photonic Network Communications* journal. He served as an Editor for IEEE INTERNET OF THINGS JOURNAL, and IEEE TRANSACTIONS ON CLOUD COMPUTING. He served as a TPC Co-Chair for several conferences including IEEE GLOBECOM 2019 (ONS) and IEEE ICC 2008 (ONS).